MODULE 12 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Decoupling with SQS, SNS and EventBridge

Queue, topic or bus - three shapes of message passing, and the clause in each scenario that picks one.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 12 of 34

admin@sthithapragna.com

A queue is work to be done. A topic is news to be broadcast. A bus is news that gets routed.

The three at a glance

	SHAPE AND DELIVERY
Amazon SQS	Queue. Consumers pull. One consumer per message. Messages persist until deleted
Amazon SNS	Topic. Push to all subscribers at once. Not persisted beyond retries
Amazon EventBridge	Bus. Rules match events to targets. Many AWS and SaaS targets, plus replay

ON THE EXAM

Persistence is the first real difference. SQS holds a message until somebody deletes it; SNS and EventBridge push and are done.

Amazon SQS fundamentals

- Producers send, consumers **poll**. Nothing is pushed to a consumer.
- A message is **hidden** during the visibility timeout, then reappears if not deleted.
- **Deleting is the consumer's job.** Processing a message does not remove it.
- Retention is **4 days by default**, configurable from 1 minute to 14 days.
- Standard queues buffer effectively unlimited throughput and scale with no configuration.

> ON THE EXAM
>
> The delete step is where bugs live. A consumer that processes successfully but never calls DeleteMessage will reprocess the same message forever.

SQS and Lambda together

- An **event source mapping** polls the queue and invokes your function with a batch.
- Lambda **deletes messages for you** when the invocation succeeds. You do not call DeleteMessage.
- On failure, nothing is deleted and the whole batch returns after the visibility timeout.
- **ReportBatchItemFailures** narrows that to only the records that actually failed.
- **Maximum concurrency** on the mapping caps consumption without throttling and re-queuing.

> ON THE EXAM
>
> This is the one case where you do not delete messages yourself. Manual DeleteMessage calls inside a Lambda SQS consumer are a sign something is misunderstood.

The SQS numbers

SETTING	VALUE
Message size	256 KB on the exam. See the currency note shortly
Retention	1 minute to 14 days, default 4 days
Visibility timeout	0 seconds to 12 hours, default 30 seconds
Long polling wait	Up to 20 seconds
Delay queue and message timer	Up to 15 minutes
FIFO throughput	300 TPS per API action, 3,000 with batching
FIFO deduplication window	5 minutes

ON THE EXAM

The 30-second default visibility timeout is shorter than many Lambda timeouts, which is exactly how duplicate processing starts. Module 5 covered the 6x rule.

Standard or FIFO queue?

STANDARD

Throughput

- Nearly unlimited throughput
- Best-effort ordering only
- At-least-once delivery
- Per-message delay supported
- The default choice

FIFO

Order and exactly-once

- 300 TPS, or 3,000 with batching
- Strict order within a message group
- Exactly-once within 5 minutes
- Queue-level delay only
- Name must end in .fifo

THE TIEBREAKER

FIFO costs you two orders of magnitude of throughput. Only choose it when ordering or duplicate suppression is a stated requirement.

FIFO mechanics

- **MessageGroupId** defines the ordering scope. Order is guaranteed **within** a group, not across groups.
- Different groups are processed in **parallel**, so group count is your concurrency.
- One group for everything means a concurrency of exactly one. A classic design mistake.
- **MessageDeduplicationId** suppresses duplicates for 5 minutes.
- **Content-based deduplication** derives that ID from a SHA-256 hash of the body instead.

> ON THE EXAM
>
> Message group ID is the single most important FIFO decision. Use the natural unit of ordering - a customer, an account, a document - not a constant.

Long polling

- **Short polling** returns immediately, often empty. You pay for every empty receive.
- **Long polling** waits up to **20 seconds** for a message to arrive before returning.
- Fewer empty responses means lower cost **and** lower latency, which is unusual.
- Set **ReceiveMessageWaitTimeSeconds** on the queue, or WaitTimeSeconds per request.
- Lambda event source mappings use long polling automatically.

> ON THE EXAM
>
> AWS recommends long polling in nearly every case. When a scenario complains about SQS API costs from an idle queue, this is the answer.

Delays and dead-letter queues

- A **delay queue** hides every new message for up to 15 minutes.
- A **message timer** does the same for one message. Standard queues only.
- A **DLQ** is an ordinary queue named in the source queue's **redrive policy**.
- **maxReceiveCount** decides how many failed receives before a message moves there.
- **Redrive to source** moves messages back once the bug is fixed, without custom code.

> ON THE EXAM
>
> A DLQ must be the same type as its source: a FIFO queue needs a FIFO dead-letter queue. Give the DLQ a longer retention than the source, or you lose evidence.

Currency note: payloads grew to 1 MB

- **The examinable answers:** SQS and SNS messages cap at **256 KB**; Lambda async payloads at **256 KB**.
- **4 August 2025:** SQS raised the maximum message payload to **1 MiB**, standard and FIFO.
- **January 2026:** Lambda asynchronous invocation payloads rose to **1 MB**, including from SQS and EventBridge.
- That removes the usual workaround of storing the body in S3 and sending a pointer.
- Billing still meters per **64 KB**, so a 1 MiB message bills as roughly 16 requests.

> READ THIS TWICE
>
> Note the last line. The limit moved, the pricing model did not, so a large message is not one cheap request. Answer 256 KB on the exam.

Amazon SNS

- Publishers send to a **topic**. Every subscription receives a copy, **pushed**.
- Subscribers: SQS, Lambda, HTTP/S, email, SMS, mobile push, Kinesis Data Firehose.
- Messages are **not stored**. A subscriber that is unavailable relies on the retry policy.
- A **subscription DLQ** catches messages that fail delivery to one subscriber.
- **FIFO topics** exist, and they can only deliver to **FIFO SQS queues**.

> ON THE EXAM
>
> Because SNS does not store messages, a subscriber that is down for an hour loses them. That single fact is why SNS is so often paired with SQS.

The fan-out pattern

- Publish once to a topic, with an **SQS queue subscribed for each consumer**.
- Each consumer gets its own queue, its own backlog and its own retry behaviour.
- A slow or failed consumer cannot affect any of the others.
- Messages persist in each queue, so a consumer can be down and catch up later.
- Adding a consumer means adding a queue. The publisher never changes.

> ON THE EXAM
>
> This is the single most examinable pattern in the module. SNS provides the fan-out; SQS provides the durability that SNS lacks.

Access control on queues and topics

- Queues and topics each have a **resource policy** controlling who may send or subscribe.
- That is how an S3 bucket, an SNS topic or another account is permitted to publish.
- Subscribing a queue to a topic needs a queue policy allowing that topic to send. It is a common omission.
- Encryption at rest uses SQS-managed keys by default, or a KMS key you choose.
- With a customer managed key, every publisher and consumer also needs **kms:Decrypt**.

> ON THE EXAM
>
> A fan-out where one queue silently receives nothing is nearly always a missing queue policy, not a broken subscription.

SNS message filtering

- A **filter policy** on a subscription decides which messages that subscriber receives.
- Filters can match **message attributes**, or the **message body** itself.
- Non-matching messages are simply not delivered, so the subscriber is never invoked.
- One topic can then serve many consumers with different interests.
- Without filtering, every subscriber receives everything and discards what it does not want.

> ON THE EXAM
>
> The same economics as event filtering on a Lambda event source mapping. Filtering at the topic means the consumer is never invoked, so you never pay for it.

Amazon EventBridge

- Events land on a **bus**. The **default bus** already receives events from most AWS services.
- **Custom buses** carry your own application events. **Partner buses** carry SaaS events.
- **Rules** match an **event pattern** against the event and send it to up to 5 targets.
- Patterns match on any field, with prefix, numeric, anything-but and exists matching.
- **Input transformers** reshape an event before it reaches the target.

> ON THE EXAM
>
> The default bus is what makes EventBridge distinctive: AWS services already publish to it, so you can react to service events without instrumenting anything.

Writing an event pattern

- A pattern is JSON that mirrors the event's shape. Fields you omit are ignored.
- Values are given as **arrays**, and any element matching counts as a match.
- Content filters: **prefix**, **suffix**, **numeric**, **cidr**, **exists** and **anything-but**.
- AWS events all carry **source**, **detail-type** and **detail**, so most rules start with source.
- A pattern that matches nothing fails silently. There is no error, just no invocations.

> ON THE EXAM
>
> The silent failure is the practical trap. Use the console's test-pattern feature against a real event rather than assuming a rule matches.

What EventBridge adds

- **Over 20 target types**, including Lambda, SQS, SNS, Step Functions, ECS and API destinations.
- **API destinations** call external HTTP endpoints, with managed auth and rate limiting.
- **Archive and replay** stores matching events and replays them later. SNS cannot do this.
- **Schema registry** discovers event structures and generates code bindings.
- At-least-once delivery, retried for up to 24 hours. A target DLQ catches what fails.

> ON THE EXAM
>
> Archive and replay is the clearest single differentiator against SNS. If a scenario needs to reprocess past events, that is an EventBridge answer.

Scheduling with EventBridge

- **Scheduled rules** on a bus fire on a rate or cron expression. The original approach.
- **EventBridge Scheduler** is a separate, purpose-built service for schedules.
- It supports **one-time** schedules, time zones, daylight saving, and flexible time windows.
- It scales to millions of schedules, and does not consume bus rule quota.
- AWS recommends Scheduler for new work. Scheduled rules remain fully supported.

> ON THE EXAM
>
> 'Run this once at a specific future time, per customer' is Scheduler. A scheduled rule can only do recurring cron on a shared bus.

SNS or EventBridge?

CHOOSE SNS WHEN

Simple fan-out, low latency

- You control both ends
- Fan-out to a known set of subscribers
- Lowest latency of the two
- Email, SMS or mobile push targets
- Very high throughput per topic

CHOOSE EVENTBRIDGE WHEN

Routing, integration, replay

- Reacting to AWS service events
- Complex content-based routing rules
- Many target types, or SaaS partners
- Archive and replay of past events
- Schema discovery and registry

THE TIEBREAKER

Email, SMS and mobile push are SNS-only. Archive, replay, AWS service events and API destinations are EventBridge-only. Those four decide most questions.

Which one does the scenario want?

THE SCENARIO SAYS	THE INTENDED ANSWER
Buffer work, smooth a spike, one worker per item	SQS
Several independent consumers of the same event	SNS fan-out to SQS queues
Order matters, or duplicates must be suppressed	SQS FIFO
React to an AWS service event	EventBridge
Route on event content to different targets	EventBridge
Replay past events after fixing a bug	EventBridge archive and replay
Send an email, an SMS or a push notification	SNS

ON THE EXAM

Worth screenshotting. Most messaging questions on this exam reduce to one row here.

When none of the three is right

- **Multiple consumers must each read every record, and replay it:** that is Kinesis, not SQS.
- **Existing RabbitMQ or ActiveMQ code you cannot change:** that is Amazon MQ.
- **Managed Kafka with existing Kafka tooling:** that is Amazon MSK.
- **Multi-step workflow with branching, retries and human approval:** that is Step Functions.
- **One service reads a queue and writes to another service:** that is EventBridge Pipes.

> ON THE EXAM
>
> Amazon MQ exists almost entirely for lift-and-shift. If a scenario mentions an existing message broker and open protocols like AMQP or MQTT, that is the signal.

Ordering and duplicates, across all three

- **SQS standard:** at-least-once, best-effort ordering. Duplicates are expected.
- **SQS FIFO:** ordered within a group, exactly-once inside a 5-minute window.
- **SNS standard:** no ordering guarantee. FIFO topics deliver only to FIFO queues.
- **EventBridge:** at-least-once, no ordering guarantee at all.
- So idempotent consumers are required almost everywhere. Module 5's pattern applies here too.

> ON THE EXAM
>
> Only SQS FIFO offers ordering, and only within a message group. Any scenario needing global ordering across everything is describing a design problem.

Monitoring a queue

- **ApproximateNumberOfMessagesVisible** is the backlog. Rising means consumers cannot keep up.
- **ApproximateAgeOfOldestMessage** is the best single health signal for a queue.
- **ApproximateNumberOfMessagesNotVisible** is work in flight.
- **NumberOfMessagesSent** against **NumberOfMessagesDeleted** shows whether you are draining.
- Alarm on the **DLQ having any messages at all**. A non-empty DLQ is always worth knowing about.

> ON THE EXAM
>
> Age of oldest message beats queue depth as an alarm. A depth of 10,000 draining quickly is fine; a depth of 5 stuck for an hour is not.

Ask the same question each time: is this **work to be done**, **news to broadcast**, or **news to route**?

> When an order is placed, four independent systems must react: inventory, billing, the shipping partner integration, and an analytics pipeline. Each processes at a different speed, and the analytics consumer is frequently down for maintenance for an hour at a time.
>
> Losing an event for any consumer is unacceptable. The order service must not need changing when a fifth consumer is added later.

What architecture should the team use?

A. Publish to an SNS topic, with a separate SQS queue subscribed for each consumer

B. Publish to an SNS topic, with each consumer's Lambda function subscribed directly

C. Write to one SQS queue that all four consumers poll

D. Have the order service call each of the four systems in turn

◆ PAUSE AND DECIDE

What architecture should the team use?

A SNS fan-out to per-consumer SQS queues

Each consumer gets its own durable backlog, so the analytics outage is survivable. Adding a fifth consumer means adding a queue, not changing the publisher.

B SNS direct to Lambda

The strongest wrong answer. SNS does not store messages, so an hour of analytics downtime loses those events once retries are exhausted.

C One shared queue

A queue message is consumed by one consumer. Four consumers polling one queue means each event is processed once, by whichever gets there first.

D Direct calls

Recouples everything. The order service now knows all four systems, and a slow or failed one blocks order placement.

THE SIGNAL

'Down for an hour' and 'losing an event is unacceptable' together demand durability. SNS provides fan-out; only the queues provide the backlog.

A payments service must apply transactions to each account in the exact order they were submitted. Transactions for different accounts are entirely independent. The system handles about 800 transactions per second across roughly 50,000 accounts.

A first attempt used a FIFO queue with a single message group ID, and throughput collapsed to a few messages per second.

How should the queue be configured?

A Switch to a standard queue and sort transactions by timestamp in the consumer

B Keep one message group but increase the consumer's reserved concurrency

C Use the account ID as the MessageGroupId, so ordering is per account and groups run in parallel

D Split transactions across 50,000 separate FIFO queues, one per account

◆ PAUSE AND DECIDE

How should the queue be configured?

A Standard queue plus sorting

A consumer cannot sort messages it has not received yet. Sorting within a batch does not give ordering across batches.

B More concurrency, one group

The exact misunderstanding that caused the problem. One message group is processed strictly one message at a time, whatever concurrency you configure.

C Account ID as MessageGroupId

Ordering is guaranteed within a group and groups run in parallel. 50,000 accounts gives 50,000 independent ordering scopes, easily covering 800 TPS.

D 50,000 queues

It would preserve ordering, and it means 50,000 queues to create, monitor and consume. Message groups exist precisely to avoid this.

THE SIGNAL

The requirement is order per account, not global order. Message group ID is how you express exactly that scope, and it is the whole reason the feature exists.

> A platform team wants a central place for domain events from twelve microservices. Different teams subscribe to different event types, and rules must route on fields inside the event body, not just a type name. Some events must also reach a third-party webhook.
>
> After a recent consumer bug, the team also needs to reprocess three days of past events once the fix ships.

Which service should carry these events?

A One SNS topic per event type, with consumers subscribing to the topics they need

B An EventBridge custom bus, with rules per consumer, API destinations, and an archive

C One SQS queue per consumer, with each service writing to every relevant queue

D An SNS topic with filter policies on each subscription

◆ PAUSE AND DECIDE

Which service should carry these events?

A. A topic per event type

Becomes unmanageable quickly, and it still cannot replay past events or call an external webhook with managed auth.

B. EventBridge custom bus

Content-based rules, API destinations for the webhook, and archive and replay for the three-day reprocess. All three requirements, one service.

C. A queue per consumer

Recouples the producers, which now must know every consumer. Adding a consumer means changing twelve services.

D. SNS with filter policies

The closest wrong answer. Filtering works, but SNS has no archive and no replay, so the three-day reprocess is impossible.

THE SIGNAL

'Reprocess three days of past events' is the disqualifying clause. Only EventBridge archives events, so only EventBridge can replay them.

A Lambda function consumes an SQS queue. Its own timeout is 60 seconds and processing usually takes about 45 seconds. The queue's visibility timeout is 30 seconds and maxReceiveCount is 5.

Downstream systems report that some records are created five times over. The configured dead-letter queue is receiving messages steadily, even though CloudWatch shows almost no Lambda errors.

What is the cause?

A maxReceiveCount is too low and should be increased to 10

B The Lambda function is throwing errors that are not being logged

C The DLQ is misconfigured and is receiving messages that succeeded

D The visibility timeout is shorter than the processing time, so messages reappear and are redelivered while still being processed

◆ PAUSE AND DECIDE

What is the cause?

A **Raise maxReceiveCount**

Would produce ten duplicates instead of five. It raises the threshold without addressing why messages are being received repeatedly.

B **Unlogged errors**

An unhandled Lambda error is always recorded in the Errors metric. The scenario says there are almost none.

C **Misconfigured DLQ**

A DLQ receives what the redrive policy sends it. It has no mechanism for capturing successful messages.

D **Visibility timeout too short**

45 seconds of processing under a 30-second visibility timeout means the message reappears mid-flight and is redelivered, five times, then moves to the DLQ.

THE SIGNAL

Five duplicates and maxReceiveCount of 5 is not a coincidence - it is the tell. AWS recommends visibility timeout of 6x the function timeout, so 360 seconds here.

What to carry forward

- **Work to be done is SQS. News to broadcast is SNS. News to route is EventBridge.**
- **SNS fan-out into SQS queues** is the most examinable pattern here. SNS does not store.
- **MessageGroupId is your FIFO concurrency.** Use the natural unit of ordering.
- Archive and replay, AWS service events and API destinations are EventBridge-only.
- Only SQS FIFO guarantees ordering, and only within a group. Assume duplicates elsewhere.

> ON THE EXAM
>
> Next: Step Functions, which orchestrates the components these three services connect.